Reviewer Pack — Real Rank of SOS Moment Matrices for Max-CUT Bounded by Degr…

Entry 3fc630b9a37e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 18:08:03 UTC

Real Rank of SOS Moment Matrices for Max-CUT Bounded by Degree

Entry ID: `3fc630b9a37e` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-12 18:08:03 UTC

1. Conjecture

Field A (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

> For a random graph G(n, 1/2) with n ≤ 40, the real rank of its degree-d SOS moment matrix M_d satisfies rank(M_d) ≤ d^2. If this fails, then the integrality gap of the d-round Lasserre hierarchy for Max-CUT on G exceeds 0.878.

Rationale (proposer's reasoning):

> The real rank of moment matrices encodes the algebraic structure of polynomial certificates. By linking it to SOS degree, we expose how geometric constraints on positive semidefiniteness constrain approximation ratios, potentially revealing barriers to higher-degree hierarchies.

Taxonomy category: `SOS_DEGREE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f93b57b11d86eca1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    n = 40
    d_values = range(2, 11)
    G = generate_random_graph(n, seed)

    M_d_values = []
    for d in d_values:
        M_d = degree_sos_moment_matrix(G, d)
        rank_M_d = real_rank(M_d)
        M_d_values.append((d, rank_M_d))

        if rank_M_d > d**2:
            return {
                "metric_name": "real_rank",
                "metric_value": rank_M_d,
                "instances_tested": len(d_values),
                "conjecture_holds": False,
                "counterexample": f"rank(M_{d}) = {rank_M_d} > {d**2}"
            }

    return {
        "metric_name": "real_rank",
        "metric_value": sum(rank for _, rank in M_d_values) / len(d_values),
        "instances_tested": len(d_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

def generate_random_graph(n: int, seed: int) -> list:
    random.seed(seed)
    G = [[0] * n for _ in range(n)]
    for i, j in combinations(range(n), 2):
        if random.random() < 0.5:
            G[i][j] = G[j][i] = 1
    return G
```

```
def degree_sos_moment_matrix(G: list, d: int) -> list:
    n = len(G)
    M_d = [[0] * (n + d) for _ in range(n + d)]

    for i in range(n):
        for j in range(i, n):
            if G[i][j]:
                for k in range(d + 1):
                    M_d[i][k] += math.comb(k, 2)
                    M_d[j][k] += math.comb(k, 2)
                    M_d[n + i][n + j] += math.comb(k, 2)

    return M_d

def real_rank(M: list) -> int:
    n = len(M)
    U = [list(row) for row in M]
    rank = 0

    for col in range(n):
        pivot_row = next((i for i in range(col, n) if U[i][col] != 0), None)
        if pivot_row is not None:
            rank += 1
            U[pivot_row], U[col] = U[col], U[pivot_row]
            for row in range(n):
                if row != col:
                    factor = -U[row][col] / U[col][col]
                    for j in range(n + len(U[0])):
                        U[row][j] += factor * U[col][j]

    return rank

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 53))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r['metric_value'] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r['conjecture_holds'] and r['counterexample'] != "" for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r['conjecture_holds'] a
        print(f"RESULT: FALSIFIED counterexample=\"{next(r['counterexample'] for r in results if r
    else:
        print(f"RESULT: INCONCLUSIVE no seeds supported the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_40a0efbf.py", line 91, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_40a0efbf.py", line 24, in run_trial
    M_d = degree_sos_moment_matrix(G, d)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_40a0efbf.py", line 63, in degree_sos_mo
    M_d[n + i][n + j] += math.comb(k, 2)
    ~~~~~~~~~~^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed before collecting data, preventing verification of conjecture | next: Fix the IndexError in degree_sos_moment_matrix implementation and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108196
2	propose	ollama_remote	qwen3:8b	0	0	121661
3	propose	ollama_remote	qwen3:8b	0	0	114578
4	propose	ollama_remote	qwen3:8b	0	0	41073
5	preregistration	ollama_remote	qwen3:8b	0	0	24082
6	novelty	ollama_remote	qwen3:8b	0	0	22670
7	novelty	ollama_remote	qwen3:8b	0	0	19660
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15239
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11508
10	judge	ollama_remote	qwen3:8b	0	0	14007

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 492673 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/3fc630b9a37e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3fc630b9a37e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3fc630b9a37e.tar.gz` (if generated)